

# Agentic AI 기반 클라우드 비용 이상 징후 탐지 및 자율 복구 시스템



## *Detection*

|         |                                                                                                                          |
|---------|--------------------------------------------------------------------------------------------------------------------------|
| 소속      | 부산대학교 정보컴퓨터공학부                                                                                                           |
| 팀명   분과 | Detection (Team.27)   네트워크/통신(C 분과)                                                                                      |
| 팀원      | 202355594 허소영 (thdual429@pusan.ac.kr)<br>202355514 강지원 (kjlw497@pusan.ac.kr)<br>202355540 박소영 (qkrthdud1309@pusan.ac.kr) |
| 지도교수    | 김태운 교수님                                                                                                                  |

# 목차

|                               |    |
|-------------------------------|----|
| 1. 연구 주제.....                 | 3  |
| 2. 요구조건 및 제약사항 수정사항.....      | 3  |
| 2.1 Gemini API 호출 제한.....     | 3  |
| 2.2 LLM에 비결정성에 따른 오분류.....    | 3  |
| 3. 설계 상세화 및 변경 내역.....        | 4  |
| 3.1 전체 파이프라인 구조 변경 요약.....    | 4  |
| 3.2 Detection Agent.....      | 5  |
| 3.3 Classification Agent..... | 6  |
| 3.4 Decision Agent.....       | 6  |
| 3.5 Action Agent.....         | 7  |
| 3.6 QA Agent.....             | 8  |
| 3.7 Logging Agent.....        | 8  |
| 4. 갱신된 과제 추진 계획.....          | 9  |
| 4.1 개발 일정 타임라인.....           | 9  |
| 4.2 다음 단계 우선순위.....           | 10 |
| 5. 구성원별 진척도.....              | 11 |
| 6. 수행 내용 및 중간 결과.....         | 11 |
| 6.1 시나리오 실험 결과.....           | 11 |

# 1. 연구 주제

**주제: Agentic AI 기반 클라우드 비용 이상 징후 탐지 및 자율 복구 시스템**

본 과제의 목표는 클라우드 재무 운영 방법론인 FinOps(Financial Operations) 관점에서, Agentic AI 아키텍처를 활용해 클라우드 환경의 비용 이상 징후를 실시간으로 탐지하고 자율적으로 복구하는 운영 시스템을 설계 및 구현하는 것이다.

Detection → Classification → Decision → Action → QA → Logging

## 2. 요구조건 및 제약사항 수정사항

착수보고서 대비 전체적인 기획 방향에는 변경이 없으며, 구현 및 테스트 과정 중, 착수 보고서에서 예상했던 제약사항(LLM 환각, AWS API Rate Limit, 액션 비가역성) 외에, 추가로 발견된 제약사항은 다음과 같다.

### 2.1 Gemini API 호출 제한

#### 1. 제약 사항

시나리오 반복 실행 시 하루 요청 제한(20회)에 걸려 429 RESOURCE\_EXHAUSTED 오류가 발생할 수 있음을 확인하였다.

#### 2. 수정 사항

반복 실험이 필요한 경우 룰 기반 fallback 경로(Decision Agent의 RULE\_BASED\_SCORE\_TABLE)를 추가하여 LLM 없이도 파이프라인이 동작 가능하도록 설계하였다.

### 2.2 LLM에 비결정성에 따른 오분류

#### 1. 제약 사항

테스트 결과, EC2의 CPU와 네트워크가 동시에 급증하는 케이스는 단순 트래픽 폭증과 DDoS 초기 징후가 혼재되어 있을 가능성이 있는 것으로 확인되었다.

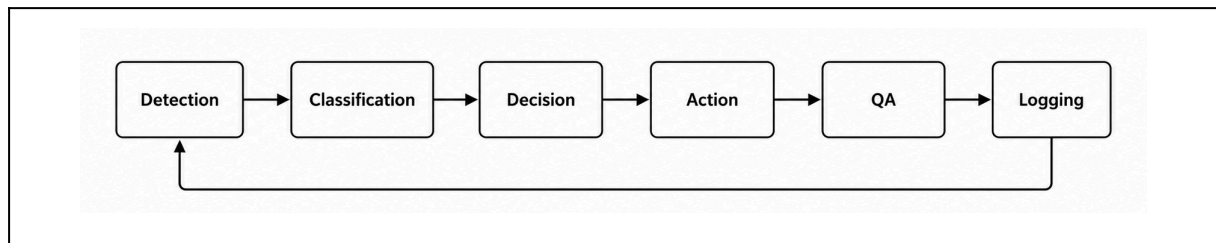
## 2. 수정 사항

CPU, 네트워크 동시 급증 케이스는 Rule-based로 고정 분류하지 않고 LLM 판단에 위임하되, risk\_security가 남발되지 않도록 Classification Agent 프롬프트에 판단 우선순위를 명시하여 방지하였다.

## 3. 설계 상세화 및 변경 내역

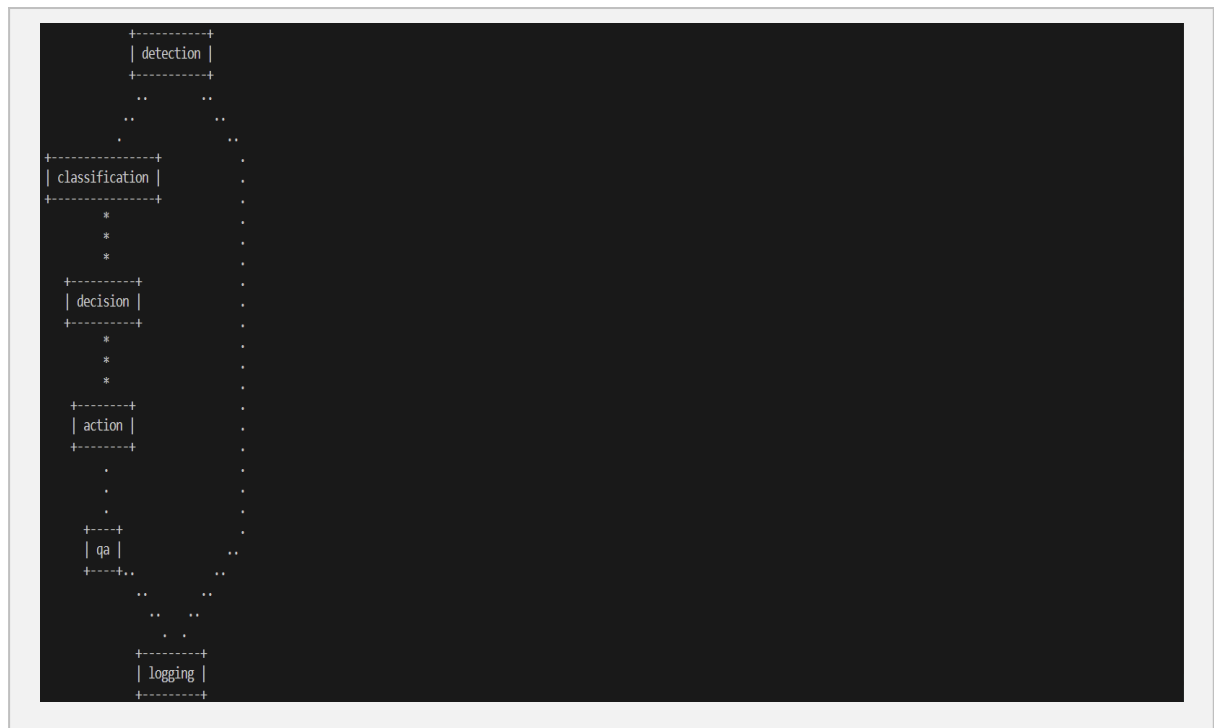
### 3.1 전체 파이프라인 구조 변경 요약

착수보고서 시스템 구성도(파이프라인 다이어그램)는 실제 구현에서 pipeline/graph.py의 LangGraph StateGraph로 그대로 구체화되었다. 조건부 엣지 구조는 다음과 같이 확정되었다.



(qa\_passed=True 이거나 rollback\_count>=2일 때 logging, qa\_passed=False & rollback\_count<2면 action 재시도)

착수보고서와의 핵심 차이는, qa\_router가 착수보고서에 기술된 "실패 시 롤백 후 재시도"를 그래프 엣지 레벨의 순환 구조(qa → action → qa)로 명시적으로 구현했다는 점이다. build\_graph(qa\_node\_override=None) 형태로 QA 노드를 교체할 수 있게 만들어, 테스트 시 강제 실패/강제 통과 노드를 주입해 롤백 루프를 검증할 수 있도록 확장하였다.



### 3.2 Detection Agent

| 항목               | 착수보고서                                      | 실제 구현                                                                                                                                                                                                                                                                       |
|------------------|--------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 데이터 수집           | CloudWatch(1분)/Cost Explorer(1시간)에서 실시간 수집 | <ul style="list-style-type: none"> <li>- state["raw_metrics"]를 그대로 사용(AWS 미연동)</li> <li>- 코드 상단 주석에 이 상태와 향후 교체 계획을 명시</li> </ul>                                                                                                                                           |
| Z-score          | k=3.0,<br>비용/네트워크입력/호출횟수 대상                | <ul style="list-style-type: none"> <li>- k=3.0 명시</li> <li>- <math>\epsilon=1e-9</math> 명시</li> <li>- 대상 지표: cost, network_in(EC2), invocation_count(Lambda), number_of_requests(S3) 적용.</li> </ul>                                                                         |
| Isolation Forest | $\tau=0.6$ , 24시간 재학습                      | <ul style="list-style-type: none"> <li>- <math>\tau=0.6</math></li> <li>- contamination=0.1</li> <li>- random_state=42</li> <li>- models/iforest_{resource_type}.pkl에 리소스 타입별 캐시</li> <li>- 24시간 경과 시 재학습</li> <li>- 재학습 데이터는 실시간 윈도우로 대체 (AWS 미연동이라 현재 윈도우로 학습)</li> </ul> |
| 결합 방식            | Z-score OR Isolation Forest                | 동일(OR 양상불) 그대로 구현                                                                                                                                                                                                                                                           |

| 항목        | 착수보고서 | 실제 구현                                                                                   |
|-----------|-------|-----------------------------------------------------------------------------------------|
| 데이터 부족 처리 | 명시 없음 | MIN_POINTS_FOR_IFOREST=5 미달 시 학습을 보류하고<br>이상 점수 0.0 처리 (임계값 5는 임의 설정, 실데이터 기반<br>조정 필요) |

### 3.3 Classification Agent

Rule-based 선처리 후 모호한 케이스만 LLM에 위임하는 부분에 대해 구현 단계에서 다음과 같이 리소스 타입별 규칙을 구체화했다.

- AutoScaling: 인스턴스 수가 평균 대비 2배 이상 급증 → risk\_security
- Lambda: invocation\_count와 error\_count가 동시에 트리거 → cost\_spike
- S3: bytes\_downloaded 단독 트리거 → risk\_security
- EC2/RDS: cost 단독 트리거 → cost\_inefficiency
- 위 규칙에 해당하지 않는 모호한 경우 → Gemini 2.5 Flash(temperature=0.1)로 위임

LLM 프롬프트에는 판단 우선순위 규칙도 함께 추가했다. risk\_security는 명확한 보안 근거가 있을 때만 사용하도록 하고, 애매한 경우에는 cost\_spike > cost\_inefficiency 순으로 보수적으로 판단하도록 지시한다. 이는 LLM이 보안 위협을 과도하게 판정해 불필요한 고위험 대응이 발생하는 것을 막기 위한 보완 장치이다. 이 외에도 JSON 파싱 실패 시 마크다운 코드블록을 제거한 뒤 재파싱하고, 최대 3회까지 재시도하는 로직을 구현해 응답 안정성을 확보했다.

### 3.4 Decision Agent

점수 공식  $\text{Score} = 0.5 \times \text{SavingRate} - 0.3 \times \text{ImpactScore} + 0.2 \times \text{StabilityScore}$ 는 착수보고서와 동일하게 구현되었다. ( \*ImpactScore : cpu 사용률 기준으로 하는 서비스에 미치는 영향도, \*StabilityScore : 입력 트래픽 수준 기준으로 하는 시스템이 안정적으로 유지될 가능성) 다만 점수 산정 방식에서 다음과 같은 구체화가 이루어졌다.

- **SavingRate를 cost 데이터 기반 결정론적 계산으로 통일**

초기 구현은 EC2만 CPU 사용률 기반 공식으로 saving\_rate를 계산하고 나머지 리소스는 LLM이 saving\_rate까지 통째로 추정하는 구조였다. 그러나 raw\_metrics에는 모든 리소스

타입 공통으로 cost 시계열이 이미 존재하여 전 리소스에 cost 기반 결정론적 계산을 기본으로 적용하도록 개선했다.

- Stop : 평균 비용의 100% 제거로 간주
- Stop+Schedule: 평균 비용의 50% 제거로 간주 (업무시간 외 절반 운영을 가정한 단순화)
- Resize: EC2 온디맨드 단가표(ap-northeast-2 리전, t3.micro~t3.xlarge 근사치)에서 현재 평균 비용과 가장 가까운 tier를 역추정하고, 한 단계 낮은 tier와의 실제 단가 차이로 계산
- Throttle/ScaleDown: cost 윈도우를 기준선 구간과 최근 급증 구간으로 나눠 그 차이(초과분)를 절감 가능액으로 계산
- Block: 비용 절감이 아니라 보안 위험 차단이 목적인 액션이므로 saving\_rate를 인위적으로 산정하지 않고 0.0으로 고정
- LLM 추정은 cost 데이터가 없거나 부족한 예외 상황에서만 saving\_rate 산정을 보조하도록 후순위로 남겼다. impact\_score/stability\_score는 여전히 정량화가 어려운 값이라 리소스 타입에 관계없이 LLM에 위임한다.
- 각 후보 액션에는 saving\_rate(비율) 외에 estimated\_saving\_usd(시간당 USD 절감 예상액) 필드를 추가해, "이 조치로 실제 얼마가 절감되는가"를 정량적으로 함께 제시할 수 있도록 했다.
- 한계: EC2 단가표는 AWS Pricing API가 아닌 정적 근사치이며, Resize의 현재 인스턴스 타입도 실제 조회값이 아니라 cost 평균으로부터의 역추정치다. AWS 연동 이후 Action Agent가 이미 스냅샷으로 저장하는 실제 인스턴스 타입으로 교체할 계획이다.

#### ● 룰 기반 폴백 테이블 추가

GEMINI\_API\_KEY가 없거나 LLM 호출이 최종 실패하는 경우를 위해 RULE\_BASED\_SCORE\_TABLE(액션별 고정 (saving, impact, stability) 값)을 도입했다. LLM 환각 대응의 연장으로, LLM이 완전히 사용 불가능한 상황까지 커버하는 3차 방어선에 해당한다.

- 환각 방어 4종(클램핑 [0,1], JSON 재시도 MAX\_LLM\_RETRIES=2, ALLOWED\_ACTIONS 표 기반 액션 제한, temperature=0.1 고정)은 착수보고서에서 계획한 대책과 대응되며 모두 구현 완료되었다.

### 3.5 Action Agent

| 리소스         | 착수보고서 요구 액션                             | 구현 상태                                                                                                                                                                                    |
|-------------|-----------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| EC2         | 좀비 인스턴스 Stop, 오버프로비저닝 Resize, 과부하 스케일아웃 | Stop, Resize 구현 완료(Resize는 stop → waiter 대기 → modify → start 순서로 구현). "과부하 시 스케일아웃"은 EC2 자체에는 대응 액션 후보가 없어 AutoScaling ScaleDown으로 대체된 구조(허용 액션 집합 상 EC2는 Stop/Stop+Schedule/Resize만 존재) |
| Lambda      | 동시성 제한(Throttle)                        | 구현 완료(put_function_concurrency, 기본 제한 10)                                                                                                                                                |
| AutoScaling | 최대 인스턴스 수 제한(ScaleDown)                 | 구현 완료(update_auto_scaling_group, 기본 max_size=2)                                                                                                                                          |

안전 요구사항과 관련하여, 착수보고서는 "위험도가 높은 액션 실행 전 알림 → 60분 대기 → 자동 실행"을 요구했는데, 현재 구현은 requires\_approval=True인 경우 알림 전송이나 대기 타이머 없이 즉시 {"status": "pending\_approval"}을 반환하고 종료한다. 60분 대기용 스케줄러는 추후 별도로 구현할 예정이다.

스냅샷 저장은 EC2/Lambda/AutoScaling에 대해 액션 실행 전 필수로 수행되도록 구현하였고, S3/RDS는 현재 구현 진행중이다.

### 3.6 QA Agent

SLA 준수 여부 검증 → 실패 시 롤백 후 재시도(최대 2회) → 초과 시 관리자 알림 흐름을 다음과 같이 구현하였다.

- SLA 검증 항목을 CPU(≤80%), 비용(직전 평균 대비 10% 초과 증가 시 위반), 가용성(액션 결과 상태) 3가지로 명확히 정의했다.
- Rule-based 우선 처리(NoAction/None은 통과, action\_result.status == "failed"는 즉시 실패) 후 모호한 경우만 LLM(Gemini)에 위임하는 구조를 추가했다. Classification Agent와 동일하게 Rule-based + LLM 조합을 QA에도 일관되게 적용한 것이다.
- 실행 실패와 SLA 위반의 원인을 구분하지 않고 항상 즉시 rollback\_action()을 호출한다. 원인별 분기 처리는 다음 단계 개선 과제로 남겨두었다.
- action\_result dict 스키마에 rolled\_back, rollback\_status, rollback\_detail, rollback\_reason, rollback\_attempt 필드를 동적으로 추가해, 하나의 액션이 "실행되었다가 롤백되었다"는 사실을 감사 로그에서 그대로 추적할 수 있게 했다.

### 3.7 Logging Agent

착수보고서에서 요구한 agent\_runs/agent\_steps/action\_log 3테이블 구조는 그대로 구현되었으며, DDL이 코드 내에 정의되어 있어 최초 실행 시 테이블이 자동 생성된다. .env의 PGHOST/PGPORT/PGDATABASE/PGUSER/PGPASSWORD로 접속하며, 값이 없으면 경고를 출력하고 기본값(localhost:5432 등)으로 접속을 시도한다.

착수보고서 대비 추가된 설계는 action\_log.success 판정 로직이다. \_build\_action\_record()에서 success = (status == "success") and not rolled\_back으로 계산하여, 실행 자체는 성공했지만 이후 QA 실패로 롤백된 액션은 최종 감사 로그상 "성공"으로 남지 않도록 처리했다. 이는 QA Agent의 롤백 필드 추가(3.6절)와 연동되는 설계로, 착수보고서에는 없던 세부 사항이다.

미구현/보류 항목은 다음과 같다.

- **Grafana 시각화:** 착수보고서 3.3.6/3.1.7이 요구한 실시간 시각화는 아직 구현하지 않았다. 코드 주석에 "AWS 미연동 상태라 실데이터가 없고, Grafana는 별도 서버/인프라가 필요한 운영 단계 작업"이라 명시했으며, 향후 대시보드에서 사용할 SQL을 별도 파일로 정리하여 관리할 계획이며, 현재 보고 시점에서는 아직 작성되지 않았다.
- **agent\_steps.duration\_ms:** 현재는 항상 NULL로 기록된다. 각 에이전트 노드가 시작/종료 시각을 state에 기록하지 않아 단계별 지연 시간 측정이 아직 불가능하다.

## 4. 갱신된 과제 추진 계획

### 4.1 개발 일정 타임라인

■ 완료된 기간(~7월 2주차) ■ 향후 예정 기간

| 작업 항목            | 3월<br>3-4주 | 4월<br>1-2주 | 4월<br>3-4주차 | 5월<br>1-2주차 | 5월<br>3-4주차 | 6월<br>1-2주차 | 6월<br>3-4주차 | 7월<br>1-2주차 | 7월<br>3-4주차 | 8월<br>1-3주차 | 8월<br>4주차 | 9월 |
|------------------|------------|------------|-------------|-------------|-------------|-------------|-------------|-------------|-------------|-------------|-----------|----|
| 사전 학습 및 경쟁 기업 분석 | ■          |            |             |             |             |             |             |             |             |             |           |    |
| 요구사항 분석 및 설계     | ■          | ■          | ■           |             |             |             |             |             |             |             |           |    |

|                                                    |  |  |  |  |  |  |  |  |  |  |  |  |
|----------------------------------------------------|--|--|--|--|--|--|--|--|--|--|--|--|
| 이상 / 복구 정의 및<br>시나리오별 대응 방안<br>구체화                 |  |  |  |  |  |  |  |  |  |  |  |  |
| MVP 구현                                             |  |  |  |  |  |  |  |  |  |  |  |  |
| EC2, S3 리소스 이상<br>대응 구현                            |  |  |  |  |  |  |  |  |  |  |  |  |
| Lambda, Auto Scaling<br>이상 대응 구현                   |  |  |  |  |  |  |  |  |  |  |  |  |
| Isolation Forest 학습 및<br>적용                        |  |  |  |  |  |  |  |  |  |  |  |  |
| 파이프라인 최종<br>설계/검증 및<br>컨테이너화                       |  |  |  |  |  |  |  |  |  |  |  |  |
| AWS CloudWatch/Cost<br>Explorer 실연동                |  |  |  |  |  |  |  |  |  |  |  |  |
| Action Agent 60분 대기<br>승인 스케줄러 구현                  |  |  |  |  |  |  |  |  |  |  |  |  |
| S3 리소스 대응(퍼블릭<br>접근 차단·IP 기반 정책)<br>구현             |  |  |  |  |  |  |  |  |  |  |  |  |
| RDS 리소스 대응 구현                                      |  |  |  |  |  |  |  |  |  |  |  |  |
| Grafana 대시보드 연동                                    |  |  |  |  |  |  |  |  |  |  |  |  |
| 단계별 실행<br>시간(duration_ms) 계측<br>instrumentation 추가 |  |  |  |  |  |  |  |  |  |  |  |  |
| 유저 제공용 대시보드<br>설계                                  |  |  |  |  |  |  |  |  |  |  |  |  |
| 최종 성능 평가 및 유지<br>보수                                |  |  |  |  |  |  |  |  |  |  |  |  |
| 최종 보고서 작성 및<br>발표                                  |  |  |  |  |  |  |  |  |  |  |  |  |

## 4.2 다음 단계 우선순위

- 시나리오 실험(playground/test\_scenarios.py)을 중간보고 시연 자료로 활용하고, 실제 AWS 리소스(EC2 Stop 등)에 대한 실행 결과를 캡처하여 다음 보고서에 첨부한다.
- Docker Compose로 PostgreSQL을 재기동하고 .env 값을 컨테이너 설정과 맞춰 INSERT 성공 여부를 재검증한다.
- AWS 연동은 detection\_agent.py의 데이터 소스 교체 지점(\_get\_or\_train\_iforest, detection\_node의 state["raw\_metrics"] 참조부)만 수정하면 되도록 이미 인터페이스가 분리되어 있어, 다음 스프린트의 최우선 과제로 배정한다.

## 5. 구성원별 진척도

| 팀원  | 현재까지 진척도                                                                                                                                                                                                                                                                                                                                                            |
|-----|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 허소영 | <ul style="list-style-type: none"><li>• Decision Agent 점수 산정 로직(cost 데이터 기반 결정론적 SavingRate 계산) 구현 완료</li><li>• Action Agent의 EC2(Stop/Resize), Lambda(Throttle), AutoScaling(ScaleDown) 실행 로직과 각 리소스별 롤백 구현 완료</li><li>• playground/test_scenarios.py 시나리오 실험 스크립트 작성 완료</li><li>• 예정 : S3/RDS 액션·스냅샷·롤백, 60분 대기 승인 스케줄러</li></ul>                                 |
| 강지원 | <ul style="list-style-type: none"><li>• 초기 노드 스키마(schema/state.py) 정의</li><li>• Classification Agent(Rule-based 우선순위 규칙 포함) 구현 완료</li><li>• QA Agent(SLA 3항목 검증 + 자동 롤백 트리거) 구현 완료</li><li>• pipeline/graph.py 통합 그래프 조립 및 조건부 엣지(detection_router, qa_router) 구현 완료</li><li>• README 작성</li><li>• 예정 : 실제 CloudWatch/Cost Explorer 연동 시의 비용 절감 검증 로직 고도화</li></ul> |
| 박소영 | <ul style="list-style-type: none"><li>• Detection Agent(Z-score+Isolation Forest OR 앙상블) 구현 완료</li><li>• Logging Agent(PostgreSQL 3테이블 DDL/INSERT) 구현 완료</li><li>• playground/test_scenarios.py 시나리오 실험 및 확인 완료</li><li>• 예정 : Grafana 대시보드, duration_ms 계측, AWS 실연동 시 CloudWatch batch 조회 반영</li></ul>                                                             |

## 6. 수행 내용 및 중간 결과

### 6.1 시나리오 실험 결과

요청사항에 따라 playground/test\_scenarios.py를 작성했다. 3개 시나리오를 순차 실행하며 각 단계(입력 메트릭 요약 → Detection → Classification → Decision → Action → QA → Logging) 결과와 총 실행 시간을 출력하고, anomaly\_flag/anomaly\_type/selected\_action/qa\_passed에 대한 검증을 assert 대신 실패해도 계속 진행하는 \_safe\_assert 헬퍼로 수행한다.

| 시나리오                                          | 입력 패턴                                                                                      | 기대 결과                                                                                                                         | 실제 반영 여부                                                                                                                     |
|-----------------------------------------------|--------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------|
| 1. 좀비 리소스<br>(EC2,<br>cost_inefficiency)      | cpu_utilization/network_in/network_out을 30개 포인트 모두 낮은 값으로 고정, cost만 마지막 3개 포인트에 작은 스파이크 삽입 | anomaly_type=cost_inefficiency,<br>selected_action=Stop,<br>risk_level=LOW,<br>requires_approval=False → 실제 boto3 EC2 Stop 실행 | 반영 완료. Z-score/Isolation Forest가 변화만 탐지하는 알고리즘 특성상 완전한 flat 데이터로는 탐지가 되지 않아, cost 지표에 최소한의 스파이크를 추가하는 보정을 적용(스크립트 주석에 이유 명시) |
| 2. Lambda 호출 폭증 (cost_spike)                  | invocation_count를 마지막 5개 포인트에서 100→5000으로 급증                                               | anomaly_type=cost_spike,<br>selected_action=Throttle,<br>risk_level=MED(60분 간 관리자 승인 대기 후, 자동 실행)                             | 반영 완료                                                                                                                        |
| 3. EDoS 의심<br>(AutoScaling,<br>risk_security) | group_in_service_instances를 마지막 5개 포인트에서 2→20으로 급증                                         | anomaly_type=risk_security,<br>selected_action=ScaleDown,<br>risk_level=HIGH(관리자 승인 대기 및 자동 실행 하지 않음)                         | 반영 완료                                                                                                                        |

시나리오 1은 EC2 인스턴스를 실제로 Stop시키므로, README에 "테스트용 인스턴스로만 실행할 것"이라는 경고를 명시했다(운영 중인 인스턴스 오조작 방지). 이 스크립트는 프로젝트 루트 또는 playground/ 디렉터리 어디에서 실행해도 동작하도록 sys.path 보정 코드를 포함했고, .env를 dotenv로 로드하며, time.time()으로 시나리오별 총 실행 시간을 측정한다.

- 시나리오 1 (EC2 좀비 리소스)

EC2 CPU 사용률 3% 유지 상태에서 비용 이상을 탐지하였고, LOW risk로 자동 Stop 이 실행되었으나, SLA 비용 검증 실패(415% 증가)로 롤백이 1회 발생하였다.

- CLI 실행 결과

```
=====
시나리오: 1. 좀비 리소스 (cost_inefficiency)
=====
EC2 CPU/네트워크 사용률이 계속 낮게 유지되는 좀비 리소스 패턴.
기대 결과: anomaly_type=cost_inefficiency, selected_action=Stop, risk_level=LOW, requires_approval=False
risk_level=LOW이므로 이 시나리오만 실제 boto3 EC2 Stop이 실행된다.

[입력 메트릭 요약]
resource_type=EC2, resource_id=i-089ad21c84e4e2728
- cpu_utilization: 시작=3.00, 끝=3.00, 평균=3.00, n=30
- network_in: 시작=100.00, 끝=100.00, 평균=100.00, n=30
- network_out: 시작=80.00, 끝=80.00, 평균=80.00, n=30
- cost: 시작=0.50, 끝=3.50, 평균=0.77, n=30

[Detection 결과]
anomaly_flag      = True
anomaly_score_zscore = 3.3148
anomaly_score_iforest = 1.0
triggered_metrics = ['cost']
[PASS] anomaly_flag == True

[Classification 결과]
anomaly_type = cost_inefficiency
reasoning    = [Rule] 비용 지표만 단독 이상, 성능 지표 정상 → 좀비 리소스 또는 오버프로비저닝
interim_action_taken = None
[PASS] anomaly_type == 'cost_inefficiency' (실제: 'cost_inefficiency')

[Decision 결과]
selected_action = Stop
risk_level      = LOW
requires_approval = False
decision_reasoning = 공식 기반 saving_rate + LLM impact/stability로 후보 4개 중 'Stop' 선택 (score=0.62, risk=LOW)
후보 액션 목록:
- NoAction    score=0.200 (saving=0.00, impact=0.00, stability=1.00)
- Stop        score=0.625 (saving=0.90, impact=0.05, stability=0.95)
- Stop+Schedule score=0.510 (saving=0.90, impact=0.40, stability=0.90)
- Resize      score=0.410 (saving=0.90, impact=0.70, stability=0.85)
[PASS] selected_action('Stop')이 허용 액션 집합 ['NoAction', 'Stop', 'Stop+Schedule', 'Resize'] 안에 있는지

[Action 결과]
action_executed = Stop
action_result   = {'status': 'success', 'raw': [{'InstanceId': 'i-089ad21c84e4e2728', 'CurrentState': {'Code': 64, 'Name': 'stopping'}, 'PreviousState': {'Code': 16, 'Name': 'running'}}]}

[QA 결과]
qa_passed      = False
rollback_count = 1
sla_check_result = {'cpu_ok': True, 'cost_ok': False, 'availability_ok': True, 'detail': '비용 증가 감지 (415.2% 증가, SLA 위반)'}
[FAIL] qa_passed == True

[총 실행 시간] 84.079초

>>> 이 시나리오에서 1건의 검증 실패가 있었습니다:
- qa_passed == True
```

- Action 전 : 인스턴스 실행 상태

인스턴스 (1) 정보 최종 업데이트 날짜 2 minutes 전  연결 인스턴스 상태 ▼ 작업 ▼ 인스턴스

저장한 필터 세트   
 필터 세트 선택 ▼ Q 인스턴스를 속성 또는 (case-sensitive) 태그로 찾기

| <input type="checkbox"/> | Name              | 인스턴스 ID             | 인스턴스 상태 ▼ | 인스턴스 유형 ▼ | 상태 검사             | 가용 영역 ▼         | 퍼블릭 IPv4 D   |
|--------------------------|-------------------|---------------------|-----------|-----------|-------------------|-----------------|--------------|
| <input type="checkbox"/> | detection-test... | i-089ad21c84e4e2728 | 실행 중      | t3.micro  | 3/3 checks passed | ap-northeast-2c | ec2-15-164-9 |

인스턴스 선택

- Action 동작 후 : 인스턴스 중지 상태

인스턴스 (1) 정보 최종 업데이트 날짜 less than a minute 전  연결 인스턴스 상태 ▼ 인스턴스

저장한 필터 세트   
 필터 세트 선택 ▼ Q 인스턴스를 속성 또는 (case-sensitive) 태그로 찾기

| <input type="checkbox"/> | Name              | 인스턴스 ID             | 인스턴스 상태 ▼ | 인스턴스 유형 ▼ | 상태 검사 | 가용 영역 ▼         |
|--------------------------|-------------------|---------------------|-----------|-----------|-------|-----------------|
| <input type="checkbox"/> | detection-test... | i-089ad21c84e4e2728 | 중지 중      | t3.micro  | -     | ap-northeast-2c |

- 롤백 후 : 인스턴스 실행 상태 (원상복구)

인스턴스 (1) 정보 최종 업데이트 날짜 less than a minute 전  연결 인스턴스 상태 ▼ 인스턴스

저장한 필터 세트   
 필터 세트 선택 ▼ Q 인스턴스를 속성 또는 (case-sensitive) 태그로 찾기

| <input type="checkbox"/> | Name              | 인스턴스 ID             | 인스턴스 상태 ▼ | 인스턴스 유형 ▼ | 상태 검사 |
|--------------------------|-------------------|---------------------|-----------|-----------|-------|
| <input type="checkbox"/> | detection-test... | i-089ad21c84e4e2728 | 실행 중      | t3.micro  | 중지    |

- 시나리오 2 (Lambda 호출 폭증)

Lambda 호출이 100→5,000으로 급증하여 LLM이 트래픽 폭증으로 분류되었다. MED risk로 승인 대기(pending\_approval) 처리되어 자동 실행 없이 의사결정자 확인 경로로 분기되었다.

```

=====
시나리오: 2. Lambda 호출 폭증 (cost_spike)
=====
Lambda invocation_count가 갑자기 급증하는 패턴 (더미 함수명, 실제 호출 안 함).
기대 결과: anomaly_type=cost_spike, selected_action=Throttle, risk_level=MED
requires_approval=True → action_result는 pending_approval로 반환된다 (코드가 구현되어 있음을 보여주는 목적).

[입력 메트릭 요약]
resource_type=Lambda, resource_id=detection-test-lambda
- invocation_count: 시작=100.00, 끝=5000.00, 평균=916.67, n=30
- error_count: 시작=1.00, 끝=1.00, 평균=1.00, n=30
- duration_avg: 시작=200.00, 끝=200.00, 평균=200.00, n=30
- cost: 시작=0.10, 끝=2.00, 평균=0.42, n=30

[Detection 결과]
anomaly_flag = True
anomaly_score_zscore = 2.2361
anomaly_score_iforest = 1.0
triggered_metrics = []
[PASS] anomaly_flag == True

```

```

[Classification 결과]
anomaly_type = cost_spike
reasoning = [LLM] Lambda 호출 수가 평균 916.667에서 5000으로 급증하여 비용이 0.417에서 2.0으로 비례하여 증가했습니다. 이는
트래픽 폭증에 의한 급증형 이상 징후입니다.
interim_action_taken = Lambda 호출 소스 및 최근 배포 변경 사항을 즉시 확인
[PASS] anomaly_type == 'cost_spike' (실제: 'cost_spike')

[Decision 결과]
selected_action = ScaleDown
risk_level = MED
requires_approval = True
decision_reasoning = LLM 전체 점수 추정 후 4개 중 'ScaleDown' 선택 (score=0.57, risk=MED)
후보 액션 목록:
- NoAction score=0.200 (saving=0.00, impact=0.00, stability=1.00)
- Throttle score=0.375 (saving=0.90, impact=0.85, stability=0.90)
- Block score=0.345 (saving=0.95, impact=1.00, stability=0.85)
- ScaleDown score=0.575 (saving=0.95, impact=0.20, stability=0.80)
[PASS] selected_action('ScaleDown')이 허용 액션 집합 ['NoAction', 'Throttle', 'Block', 'ScaleDown'] 안에 있는지

[Action 결과]
action_executed = None
action_result = {'status': 'pending_approval'}

[QA 결과]
qa_passed = True
rollback_count = 0
sla_check_result = {'cpu_ok': True, 'cost_ok': True, 'availability_ok': True, 'detail': 'NoAction - 액션 없음, 검증 스킵'}
[PASS] qa_passed == True

[총 실행 시간] 58.764초

>>> 모든 검증 통과

```

- 시나리오 3 (EDoS 의심)

AutoScaling 인스턴스가 2→20으로 10배 급증하여 규칙 기반으로 EDoS 의심으로 분류되었다. 이후 HIGH risk로 즉시 최대 인스턴스 수 제한 후 승인 대기 처리되었다.

```

=====
시나리오: 3. EDoS 의심 (risk_security)
=====
AutoScaling group_in_service_instances가 갑자기 급증하는 패턴 (더미 그룹명, 실제 호출 안 함).
기대 결과: anomaly_type=risk_security, selected_action=ScaleDown, risk_level=HIGH
requires_approval=True → action_result는 pending_approval로 반환된다.

```

```

[입력 메트릭 요약]
resource_type=AutoScaling, resource_id=detection-test-asg
- group_in_service_instances: 시작=2.00, 끝=20.00, 평균=5.00, n=30
- group_desired_capacity: 시작=2.00, 끝=20.00, 평균=5.00, n=30
- cost: 시작=0.50, 끝=5.00, 평균=1.25, n=30

[Detection 결과]
anomaly_flag = True
anomaly_score_zscore = 2.2361
anomaly_score_iforest = 1.0
triggered_metrics = []
[PASS] anomaly_flag == True

[Classification 결과]
anomaly_type = risk_security
reasoning = [Rule] AutoScaling 인스턴스 수가 평균 대비 2배 이상 급증 → EDoS 의심
interim_action_taken = AutoScaling 최대 인스턴스 수 임시 제한
[PASS] anomaly_type == 'risk_security' (실제: 'risk_security')

[Decision 결과]
selected_action = ScaleDown
risk_level = HIGH
requires_approval = True
decision_reasoning = LLM 전체 점수 추정 후 3개 중 'ScaleDown' 선택 (score=0.60, risk=HIGH)
후보 액션 목록:
- NoAction score=0.200 (saving=0.00, impact=0.00, stability=1.00)
- Block score=0.340 (saving=0.90, impact=0.80, stability=0.65)
- ScaleDown score=0.600 (saving=0.90, impact=0.10, stability=0.90)
[PASS] selected_action('ScaleDown')이 허용 액션 집합 ['NoAction', 'Block', 'ScaleDown'] 안에 있는지

[Action 결과]
action_executed = None
action_result = {'status': 'pending_approval'}

[QA 결과]
qa_passed = True
rollback_count = 0
sla_check_result = {'cpu_ok': True, 'cost_ok': True, 'availability_ok': True, 'detail': 'NoAction - 액션 없음, 검증 스킵'}
[PASS] qa_passed == True

[총 실행 시간] 34.447초

```

## ● Action Log

각 시나리오 실행 결과는 실시간으로 DB에 기록된다. 상단 Action Log 테이블에는 실행된 액션명, risk\_level, 승인 여부, 실행 전 스냅샷, 액션 결과가 저장된다. 또한, agent\_runs 테이블과 action\_log 테이블을 JOIN하여 조회한 하단 요약 테이블에는 anomaly\_type, selected\_action, QA 통과 여부가 리소스 단위로 확인할 수 있다. 이를 통해 에이전트의 의사결정 이력을 추적하고 사후 감사가 가능하다.

[그림 1] action\_log 테이블 조회 결과 — 실제 액션이 실행된 경우에만 기록되며, 액션명, risk\_level, 승인 여부, 실행 전 스냅샷, 액션 결과를 포함

| AZ resource_id      | AZ action_name | AZ risk_level | requires_approval | pre_action_snapshot                                                                                                     | action_result                                                                         | success |
|---------------------|----------------|---------------|-------------------|-------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------|---------|
| test-ec2-001        | ScaleDown      | MED           | [v]               | ["state": "running", "instance_type": "t3.medium", "security": {"status": "success"}]                                   | ["state": "running", "instance_type": "t3.medium", "security": {"status": "success"}] | [v]     |
| test-ec2-001        | ScaleDown      | MED           | [v]               | ["state": "running", "instance_type": "t3.medium", "security": {"status": "success"}]                                   | ["state": "running", "instance_type": "t3.medium", "security": {"status": "success"}] | [v]     |
| test-ec2-001        | ScaleDown      | MED           | [v]               | ["state": "running", "instance_type": "t3.medium", "security": {"status": "success"}]                                   | ["state": "running", "instance_type": "t3.medium", "security": {"status": "success"}] | [v]     |
| test-ec2-001        | ScaleDown      | MED           | [v]               | ["state": "running", "instance_type": "t3.medium", "security": {"status": "success"}]                                   | ["state": "running", "instance_type": "t3.medium", "security": {"status": "success"}] | [v]     |
| i-089ad21c84e4e2728 | Stop           | LOW           | [ ]               | ["state": "running", "instance_type": "t3.micro", "security": {"raw": [{"InstanceId": "i-089ad21c84e4e2728", "Current": |                                                                                       | [ ]     |

[그림 2] agent\_runs × action\_log JOIN 조회 결과 — 전체 파이프라인 실행 이력과 액션 실행 여부를 리소스별로 확인

|   | AZ resource_type | AZ resource_id        | AZ anomaly_type   | AZ selected_action | AZ risk_level | requires_approval | action_success | qa_passed |
|---|------------------|-----------------------|-------------------|--------------------|---------------|-------------------|----------------|-----------|
| 1 | EC2              | i-089ad21c84e4e2728   | cost_inefficiency | Stop               | LOW           | [ ]               | [ ]            | [ ]       |
| 2 | AutoScaling      | detection-test-asg    | risk_security     | ScaleDown          | HIGH          | [v]               | [NULL]         | [v]       |
| 3 | Lambda           | detection-test-lambda | cost_spike        | ScaleDown          | MED           | [v]               | [NULL]         | [v]       |

*Detection · Agentic AI 기반 클라우드 비용 이상 징후 탐지 및 자율 복구 시스템*